

GUIA DE ESTUDIO

GRAMATICAS FORMALES

Material:

Navarrete, I., Cardenas, M., Sanchez, D., Botía, J., Marín, R. Martínez, R. 2008. *Teoría de Autómatas y Lenguajes Formales*. Departamento de Ingenieria de la Informacion y las Comunicaciones. Universidad de Murcia

1. Defina intuitivamente "Gramática".
 2. Defina formalmente "Gramática" y "Regla de Producción".
 3. Defina "derivación de una cadena en otra".
 4. Defina " α derivable en β ".
 5. Defina "forma sentencial" y "sentencia".
 6. Defina "Lenguaje Generado por G".
 7. Defina "equivalencia entre gramáticas".
 8. Defina "BNF".
 9. Defina "Jerarquía de Chomsky" y describa las cuatro clases.
 10. Enuncie el Teorema sobre inclusión de Lenguajes en la "Jerarquía de Chomsky".
-

En ambos casos, L^n se refiere a la potencia n -ésima del lenguaje L en el monoide inducido $(\mathcal{P}(V^*), \cdot, \{\lambda\})$. El cierre o clausura de un lenguaje, por definición, contiene cualquier palabra que se obtenga por concatenación de palabras de L y además la palabra vacía.

3. Gramáticas formales

Hasta ahora hemos descrito los lenguajes formales como se describen los conjuntos: por extensión (si son finitos) o por comprensión. Aquí vamos a introducir otra forma general y rigurosa de describir un lenguaje formal: mediante el uso de gramáticas. Las gramáticas son mecanismos *generadores* de lenguajes, es decir, nos dicen cómo podemos obtener o construir palabras de un determinado lenguaje.

3.1. Definiciones básicas

Definición 1.1 Una GRAMÁTICA es una cuadrupla $G = (V_N, V_T, S, P)$ donde:

V_T es el alfabeto de símbolos terminales

V_N es el alfabeto de símbolos no terminales o variables, de forma que debe ser $V_N \cap V_T = \emptyset$ y denotamos con V al alfabeto total de la gramática, esto es, $V = V_N \cup V_T$.

S es el símbolo inicial y se cumple que $S \in V_N$

P es un conjunto finito de reglas de producción

Definición 1.2 Una REGLA DE PRODUCCIÓN es un par ordenado (α, β) de forma que:

$$(\alpha, \beta) \in (V^* \cdot V_N \cdot V^*) \times V^*$$

Es decir, $\alpha = \gamma_1 A \gamma_2$ donde $\gamma_1, \gamma_2 \in (V_N \cup V_T)^*$, $A \in V_N$ y $\beta \in (V_N \cup V_T)^*$. Una producción $(\alpha, \beta) \in P$ se suele escribir de forma infija como $\alpha \rightarrow \beta$.

Por **convenio** usaremos letras mayúsculas para los símbolos no terminales; dígitos y las primeras letras minúsculas del alfabeto para los símbolos terminales; las últimas letras minúsculas del alfabeto para palabras que pertenezcan a V_T^* y letras griegas para cualquier palabra que pertenezca a V^* . Usando este convenio, a veces se suele describir una gramática enumerando únicamente sus reglas de producción y cuando varias reglas tienen la misma parte izquierda, se suelen agrupar separándolas con $|$.

Ejemplo 1.6 Sea la gramática G cuyas producciones son:

$$S \rightarrow aSa \mid bSb \mid a \mid b \mid \lambda$$

Esta gramática tiene una sola variable S que además es el símbolo inicial. $V_T = \{a, b\}$ y P contiene 5 reglas de producción.

Definición 1.3 Sea G una gramática y sean las cadenas $\alpha, \beta \in V^*$. Decimos que α deriva directamente en β , que notamos como $\alpha \Rightarrow \beta$ (DERIVACIÓN DIRECTA), si y sólo si existe una producción $\delta \rightarrow \sigma \in P$ tal que $\alpha = \gamma_1 \delta \gamma_2$, $\beta = \gamma_1 \sigma \gamma_2$ con $\gamma_1, \gamma_2 \in V^*$.

Esto quiere decir que α deriva directamente en β , si β puede obtenerse a partir de α sustituyendo una ocurrencia de la parte izquierda de una producción que aparezca en α por la parte derecha de la regla de producción.

Nota Si $\alpha \rightarrow \beta$ es una regla de producción de G , entonces se cumple siempre que $\alpha \Rightarrow \beta$. Cuando sea necesario distinguir entre varias gramáticas, escribiremos $\alpha \Rightarrow_G \beta$, para referirnos a una derivación directa en G .

Por la definición anterior se deduce que $\Rightarrow$ es una relación binaria en el conjunto de cadenas de la gramática, esto es: $\Rightarrow \subseteq V^* \times V^*$. Aquí usamos una notación infija para indicar que $\alpha \Rightarrow \beta$ en lugar de $(\alpha, \beta) \in \Rightarrow$.

Definición 1.4 Decimos que α deriva en β , o bien que, β es derivable de α , y lo notamos como $\alpha \Rightarrow^* \beta$ (DERIVACIÓN) si y sólo si se verifica una de las dos condiciones siguientes:

1. $\alpha = \beta$, (son la misma cadena), o bien,
2. $\exists \gamma_0, \gamma_1, \dots, \gamma_n \in V^*$ tal que $\gamma_0 = \alpha$, $\gamma_n = \beta$ y $\forall 0 \leq i < n$ se cumple que $\gamma_i \Rightarrow \gamma_{i+1}$

A la secuencia $\gamma_0 \Rightarrow \gamma_1 \Rightarrow \dots \Rightarrow \gamma_n$ la llamaremos *secuencia de derivaciones directas* de longitud n , o simplemente *derivación* de longitud n .

Nota Por la definición anterior está claro que $\Rightarrow^*$ es también una relación binaria en V^* y además es la *clausura reflexiva y transitiva* de la relación de derivación directa $\Rightarrow$. Esto quiere decir que $\Rightarrow^*$ es la *menor relación* que cumple lo siguiente:

- Si $\alpha \Rightarrow \beta$ entonces $\alpha \Rightarrow^* \beta$. Esto es, si dos cadenas están relacionadas mediante $\Rightarrow$ entonces también lo están mediante la relación $\Rightarrow^*$
- $\Rightarrow^*$ es reflexiva, ya que $\forall \alpha \in V^*$ se cumple que $\alpha \Rightarrow^* \alpha$
- $\Rightarrow^*$ es transitiva. En efecto, si $\alpha \Rightarrow^* \beta$ y $\beta \Rightarrow^* \gamma$, entonces $\alpha \Rightarrow^* \gamma$

Definición 1.5 Sea una gramática $G = (V_N, V_T, S, P)$. Una palabra $\alpha \in (V_N \cup V_T)^*$ se denomina **FORMA SENTENCIAL** de la gramática, si y sólo si se cumple que: $S \Rightarrow^* \alpha$. Una forma sentencial w tal que $w \in V_T^*$ se dice que es una **SENTENCIA**.

Ejemplo 1.7 Sea la gramática $S \rightarrow aSa \mid bSb \mid a \mid b \mid \lambda$, podemos afirmar lo siguiente:

- $aaSb \Rightarrow aabSbb$, aunque ni $aaSb$ ni $aabSbb$ son formas sentenciales de G
- $aabb \Rightarrow^* aabb$, aunque $aabb$ no es una sentencia de G
- S , aSa , $abSba$, λ son formas sentenciales de G y además λ es una sentencia
- $aabaa$ es una sentencia de G , ya que existe una derivación de longitud 3 por la que $S \Rightarrow^* aabaa$. En efecto:

$$S \Rightarrow aSa \Rightarrow aaSaa \Rightarrow aabaa$$

Definición 1.6 Sea una gramática $G = (V_N, V_T, S, P)$. Se llama **LENGUAJE GENERADO** por la gramática G al lenguaje $L(G)$ formado por todas las cadenas de símbolos terminales que son derivables del símbolo inicial de la gramática (sentencias):

$$L(G) = \{w \in V_T^* \mid S \Rightarrow^* w\}$$

Ejemplo 1.8 Sea $L = \{w \in \{a, b\}^* \mid w = w^R\}$. Este lenguaje está formado por todos los palíndromos sobre el alfabeto $\{a, b\}$. Puede probarse que la gramática $S \rightarrow aSa \mid bSb \mid a \mid b \mid \lambda$ genera el lenguaje L . En general no existe un método exacto para probar que una gramática genera un determinado lenguaje. Para este caso tan sencillo podemos probarlo de “manera informal” haciendo una serie de derivaciones hasta darnos cuenta de que $S \Rightarrow^* w$ si y sólo si $w = w^R$. Luego veremos una demostración formal por inducción en la sección de aplicaciones.

Definición 1.7 Dos gramáticas G y G' son EQUIVALENTES si y sólo si generan el mismo lenguaje, es decir, si $L(G) = L(G')$.

3.2. Notación BNF

A veces se utiliza una notación especial para describir gramáticas llamada notación *BNF* (*Backus-Naur-Form*). En la notación *BNF* los símbolos no terminales o variables son encerrados entre ángulos y utilizaremos el símbolo $::=$ para las producciones, en lugar de $\rightarrow$. Por ejemplo, la producción $S \rightarrow aSa$ se representa en *BNF* como $\langle S \rangle ::= a \langle S \rangle a$. Tenemos también la notación *BNF-extendida* que incluye además los símbolos $[]$ y $\{\}$ para indicar elementos opcionales y repeticiones, respectivamente.

Ejemplo 1.9 Supongamos que tenemos un lenguaje de programación cuyas dos primeras reglas de producción para definir su sintaxis son:

$$\begin{aligned}\langle \text{programa} \rangle &::= [\langle \text{cabecera} \rangle] \text{ begin } \langle \text{sentencias} \rangle \text{ end} \\ \langle \text{sentencias} \rangle &::= \langle \text{sentencia} \rangle \{ \langle \text{sentencia} \rangle \}\end{aligned}$$

Esto viene a decir que un programa se compone de una cabecera opcional, seguido de la palabra clave “begin”, a continuación una lista de sentencias (debe haber al menos una sentencia) y finaliza con la palabra clave “end”. Podemos transformar las producciones anteriores para especificarlas, según la notación que nosotros hemos introducido (estándar), de la siguiente forma:

$$\begin{aligned}P &\rightarrow C \text{ begin } A \text{ end} \mid \text{ begin } A \text{ end} \\ A &\rightarrow B A \mid B\end{aligned}$$

donde P es el símbolo inicial de la gramática y corresponde a la variable $\langle \text{programa} \rangle$, C corresponde a $\langle \text{cabecera} \rangle$, A se refiere a la variable $\langle \text{sentencias} \rangle$ y B a $\langle \text{sentencia} \rangle$.

La simbología utilizada para describir las gramáticas en notación estándar y en notación *BNF* nos proporcionan una herramienta para describir los lenguajes y la estructura de las sentencias del lenguaje. Puede considerarse a esta simbología como un *metalenguaje*, es decir un lenguaje que sirve para describir otros lenguajes.

3.3. Jerarquía de Chomsky

En 1959 *Chomsky* clasificó las gramáticas en cuatro familias, que difieren unas de otras en la forma que pueden tener sus reglas de producción. Si tenemos una gramática $G = (V_N, V_T, S, P)$ clasificaremos las gramáticas y los lenguajes generados por ellas, de la siguiente forma:

- TIPO 3 (*Gramáticas regulares*). Pueden ser, a su vez, de dos tipos:
 - *Lineales por la derecha*. Todas sus producciones son de la forma:

$$\begin{aligned}A &\rightarrow bC \\ A &\rightarrow b \\ A &\rightarrow \lambda\end{aligned}$$

donde $A, C \in V_N$ y $b \in V_T$.

- *Lineales por la izquierda.* Con producciones del tipo:

$$\begin{array}{l} A \rightarrow Cb \\ A \rightarrow b \\ A \rightarrow \lambda \end{array}$$

Los lenguajes generados por estas gramáticas se llaman LENGUAJES REGULARES y el conjunto de todos estos lenguajes es la clase $\mathcal{L}_3$.

- TIPO 2 (*Gramáticas libres del contexto*). Las producciones son de la forma:

$$A \rightarrow \alpha$$

donde $A \in V_N$ y $\alpha \in (V_N \cup V_T)^*$. Los lenguajes generados por este tipo de gramáticas se llaman LENGUAJES LIBRES DEL CONTEXTO y la clase es $\mathcal{L}_2$.

- TIPO 1 (*Gramáticas sensibles al contexto*). Las producciones son de la forma:

$$\alpha A \beta \rightarrow \alpha \gamma \beta$$

donde $\alpha, \beta \in V^*$ y $\gamma \in V^+$. Se permite además la producción $S \rightarrow \lambda$ siempre y cuando S no aparezca en la parte derecha de ninguna regla de producción.

El sentido de estas reglas de producción es el de especificar que una variable A puede ser reemplazada por γ en una derivación directa sólo cuando A aparezca en el “contexto” de α y β , de ahí el nombre “sensibles al contexto”. Además, las producciones de esa forma cumplen siempre que la parte izquierda tiene longitud menor o igual que la parte derecha, pero nunca mayor (excepto para $S \rightarrow \lambda$). Esto quiere decir que la gramática es *no contráctil*.

Los lenguajes generados por las gramáticas de tipo 1 se llaman LENGUAJES SENSIBLES AL CONTEXTO y su clase es $\mathcal{L}_1$.

- TIPO 0 (*Gramáticas con estructura de frase*) Son las gramáticas más generales, que por ello también se llaman *gramáticas sin restricciones*. Esto quiere decir que las producciones pueden ser de cualquier tipo permitido, es decir, de la forma $\alpha \rightarrow \beta$ con $\alpha \in (V^* \cdot V_N \cdot V^*)$ y $\beta \in V^*$.

Los lenguajes generados por estas gramáticas son los LENGUAJES CON ESTRUCTURA DE FRASE, que se agrupan en la clase $\mathcal{L}_0$. Estos lenguajes también se conocen en el campo de la Teoría de la Computabilidad como *lenguajes recursivamente enumerables*.

Teorema

1.2 (Jerarquía de Chomsky) Dado un alfabeto V , el conjunto de los lenguajes regulares sobre V está incluido propiamente en el conjunto de los lenguajes libres de contexto y este a su vez está incluido propiamente en el conjunto de los lenguajes sensibles al contexto, que finalmente está incluido propiamente en el conjunto de lenguajes con estructura de frase. Esto es:

$$\mathcal{L}_3 \subset \mathcal{L}_2 \subset \mathcal{L}_1 \subset \mathcal{L}_0$$

La demostración de este teorema la iremos viendo a lo largo del curso.

Nota

En este tema hemos hecho referencia al término *lenguaje formal* para diferenciarlo de *lenguaje natural*. En general, un lenguaje natural es aquel que ha evolucionado con el paso del tiempo para fines de la comunicación humana, por

ejemplo el español o el inglés. Estos lenguajes evolucionan sin tener en cuenta reglas gramaticales formales. Las reglas surgen después con objeto de explicar, más que determinar la *estructura* de un lenguaje, y la sintaxis es difícil de determinar con precisión. Los lenguajes formales, por el contrario, están definidos por reglas de producción preestablecidas y se ajustan con todo rigor o “formalidad” a ellas. Como ejemplo tenemos los lenguajes de programación y los lenguajes lógicos y matemáticos. No es de extrañar, por tanto, que se puedan construir compiladores eficientes para los lenguajes de programación y que por contra la construcción de traductores para lenguaje natural sea una tarea compleja e ineficiente, en general. Veremos que las gramáticas regulares y libres de contexto, junto con sus máquinas abstractas asociadas tienen especial interés en la construcción de traductores para lenguajes de programación.

4. Nociones básicas sobre traductores

Hace apenas unas cuantas décadas, se utilizaban los llamados *lenguajes de primera generación* para hacer que los computadores resolvieran problemas. Estos lenguajes operan a nivel de código binario de la máquina, que consiste en una secuencia de ceros y unos con los que se instruye al ordenador para que realice acciones. La programación, por tanto, era difícil y problemática, aunque pronto se dio un pequeño paso con el uso de código octal o hexadecimal.

El código de máquina fue reemplazado por los *lenguajes de segunda generación*, o *lenguajes ensambladores*. Estos lenguajes permiten usar abreviaturas nemónicas como nombres simbólicos, y la abstracción cambia del nivel de flip-flop al nivel de registro. Se observan ya los primeros pasos hacia la estructuración de programas, aunque no puede utilizarse el término de *programación estructurada* al hablar de programas en ensamblador. Las desventajas principales del uso de los lenguajes ensambladores son, por un lado, la dependencia de la máquina y, por otro, que son poco legibles.

Para sustituir los lenguajes ensambladores, se crearon los *lenguajes de tercera generación* o *lenguajes de alto nivel*. Con ellos se pueden usar estructuras de control basadas en objetos de datos lógicos: variables de un tipo específico. Ofrecen un nivel de abstracción que permite la especificación de los datos, funciones o procesos y su control en forma independiente de la máquina. El diseño de programas para resolver problemas complejos es mucho más sencillo utilizando este tipo de lenguajes, ya que se requieren menos conocimientos sobre la estructura interna del computador, aunque es obvio que el ordenador únicamente entiende código máquina. Por lo tanto, para que un computador pueda ejecutar programas en lenguajes de alto nivel, estos deben ser traducidos a código máquina. A este proceso se le denomina *compilación*, y la herramienta correspondiente se llama *compilador*. Nosotros vamos a entender el término *compilador* como un programa que lee otro, escrito en *lenguaje fuente*, y lo traduce a *lenguaje objeto*, informando, durante el proceso de traducción, de la presencia de errores en el programa fuente. Esto se refleja en la figura 1.1.

En la década de 1950, se consideró a los compiladores como programas notablemente difíciles de escribir. El primer compilador de FORTRAN, por ejemplo, necesitó para su implementación, 18 años de trabajo en grupo. Desde entonces, se han descubierto técnicas sistemáticas para manejar muchas de las importantes tareas que surgen en la compilación. También se han desarrollado buenos lenguajes de implementación, entornos de programación y herramientas de software. Con estos avances, puede construirse un compilador real incluso como proyecto de estudio en una asignatura sobre diseño de compiladores.